MASTER OF COMPUTER APPLICATIONS (MCA) PROJECT VIVA PRESENTATION

NK AUTOMOBILES

Vehicle Inventory and Management System with Two-Wheeler Fitment Engine

PROJECT DEVELOPERS

- **Shridhar Kusugal (USN: 2XXMCAXXX)**
- **Mohammad Kaif (USN: 2XXMCAXXX)**

Course: Master of Computer Applications (MCA)
Academic Year: 2025 – 2026

INSTITUTION & GUIDANCE

Chetan Business School
Department of MCA, Hubballi, Karnataka

Project Guide: [Prof. / Dr. Project Guide Name]
Designation: [Assistant Professor / HOD, Dept. of MCA]

PROJECT BACKGROUND

Introduction & Project Overview

Industry Context

- **Automobile Retail:** Retail automobile spare parts in India is a high-volume, multi-brand industry requiring precise fitment.
- **Hubballi Enterprise:** NK Automobiles operates as an established multi-brand two-wheeler spare parts hub in Hubballi, Karnataka.
- **Digital Shift:** Traditional paper-based ledger systems struggle with expanding catalogs, stock verification, and rapid billing.
- **Scale of Operation:** The enterprise caters to hundreds of mechanics, vehicle owners, and retail customers daily.

System Purpose

- **Unified Portal:** Centralize spare-parts inventory, sales processing, and customer requests into one responsive web portal.
- **Part Compatibility:** Eliminate part mismatch by mapping spare parts directly to verified two-wheeler models.
- **Frictionless Sales:** Enable customers and mechanics to browse 181+ verified parts and complete orders via Guest Checkout.
- **UPI Integration:** Incorporate modern dynamic UPI QR scan-to-pay workflow for instant digital payment verification.

Key Highlights

- **181 Active Products:** Spanning 8 automotive categories (Engine, Brake, Chains, Electricals, Filters, etc.).
- **1,650+ Compatibility Rules:** Authentic vehicle mapping across 11 popular Indian bike & scooter models.
- **Role-Based Security:** Separate secure interfaces for Customers and Admin management.
- **Full-Stack Architecture:** Built with lightweight Python Flask, MySQL 8.0, and responsive Bootstrap 5.

OPERATIONAL CHALLENGES

Problem Statement: Challenges in Manual Operations

1. Incompatible Part Mismatches & Returns

- **Lack of Fitment Verification:** Customers frequently purchase wrong parts due to subtle model year or emission (BS3/BS4/BS6) variations.
- **High Return Overhead:** Part returns cause inventory discrepancies, wasted counter time, and customer dissatisfaction.

2. Fragmented Stock & Sudden Stockouts

- **Manual Ledger Tracking:** Inventory records maintained in physical books or isolated spreadsheets are prone to manual logging errors.
- **Zero Low-Stock Visibility:** Shop owners discover stockouts only when a customer physically asks for a part at the counter.

3. Slow Counter Checkout & Billing Bottlenecks

- **Handwritten Invoices:** Manual billing creates long queues during peak morning and evening mechanic rush hours.
- **Lost Customer History:** No digital record of past repairs, repeat customer buying habits, or warranty tracking.

4. Payment Reconciliation & Audit Deficits

- **Disjointed Payment Tracking:** Cash transactions combined with personal UPI QR scans lead to daily cash-drawer mismatch.
- **Lack of Activity Audit:** No audit log exists to identify who altered stock quantities or issued discounts.

PROJECT GOALS

Project Objectives & Scope

01

Centralized Catalog & Inventory

Develop a relational database-backed catalog storing 181+ verified parts with real-time stock and pricing tracking.

02

Intelligent Fitment Search Engine

Implement dynamic vehicle compatibility mapping across 11 key Indian motorcycle & scooter platforms with live filtering.

03

Streamlined Customer Storefront

Provide a fast, responsive e-commerce interface with category filtering, live search, cart calculation, and Guest Checkout.

04

Integrated UPI Digital Payments

Generate dynamic, order-specific UPI QR codes allowing seamless scan-to-pay functionality on mobile and desktop.

05

Admin Control & Security

Establish role-based access control, product CRUD operations, verified image asset management, and system activity logs.

SYSTEM COMPARISON

Existing System vs. Proposed System

Evaluation Parameter	Existing Manual / Legacy System	Proposed NK Automobiles System
Inventory Tracking	Physical stock registers or static Excel sheets; stock levels frequently out of sync.	Centralized MySQL 8.0 database; automated atomic stock deduction upon order completion.
Part Compatibility	Relies entirely on counter staff memory; high rate of wrong part sales.	Engineered Two-Wheeler Compatibility Engine with 1,655+ rules and live bike search.
Customer Access	Restricted to physical storefront counter during working business hours.	24/7 web access across smartphones and desktop devices with responsive UI.
Checkout & Ordering	Handwritten bill slips; mandatory queue; no digital order record.	Frictionless Guest Checkout and authenticated portal with order tracking.
Payment Reconciliation	Loose cash handling or unverified manual UPI screenshots.	Order-tagged dynamic UPI QR codes with UTR reference validation.
Security & Auditing	Zero access control; registers vulnerable to physical loss or tampering.	Bcrypt/Werkzeug password hashing, session RBAC, and dedicated system_logs.

SYSTEM IMPLEMENTATION

Technology Stack & Development Environment

Backend Technologies

- **Language:** Python 3.12+ (Robust, efficient, highly readable)
- **Framework:** Flask (Lightweight, modular WSGI web framework)
- **Security Engine:** Werkzeug Security (SHA-256 / PBKDF2 password hashing)
- **Architecture:** Model-View-Controller (MVC) server pattern
- **Session Management:** Flask Client-side cryptographically signed sessions

Database Tier

- **RDBMS Engine:** MySQL 8.0 Community Server
- **Python Driver:** mysql-connector-python (Official Oracle driver)
- **Data Integrity:** ACID transactions with foreign key constraints
- **Query Pattern:** Strictly Parameterized SQL queries (Zero SQL injection)
- **Database Scale:** 14 relational tables, 181 products, 1,655 mappings

Frontend & User Interface

- **Core Markup:** Semantic HTML5 with SEO meta tags
- **Styling Framework:** Bootstrap 5.3 (Mobile-first responsive grid system)
- **Client Scripting:** Vanilla JavaScript (ES6+) for DOM manipulation & search
- **Template Engine:** Jinja2 (Server-side dynamic template rendering)
- **Icons & Assets:** FontAwesome 6.4 + High-res verified part photography

Tools & Environment

- **IDE / Editor:** Visual Studio Code with Python & SQL extensions
- **Local Server:** XAMPP / MySQL Service on port 3306
- **Version Control:** Git & GitHub for source code management
- **API Testing:** Postman & Python unittest suite
- **Image Processing:** Pillow (PIL) for image optimization & verification

SYSTEM DESIGN

System Architecture: 3-Tier Web Architecture

Tier 1: Presentation Layer (Client Tier)

- **User Devices:** Mobile Smartphones, Tablets, Desktop Web Browsers accessing HTTP/HTTPS endpoints.
- **Frontend Technologies:** Bootstrap 5 Responsive Grid, HTML5 Semantic Layouts, CSS3 Transitions, FontAwesome Icons.
- **Client-Side Logic:** JavaScript Event Listeners (Live vehicle search, cart item quantity adjuster, dynamic modal popups, form validation).

Tier 2: Application & Business Logic Layer (Python Flask Server)

- **WSGI Controller (app.py):** Central routing engine managing 25+ endpoints (/products, /product/<id>, /checkout, /admin).
- **Authentication & Security:** Werkzeug password hashing, session-based role checks (is_admin), CSRF protection & input sanitization.
- **Core Business Engines:** Two-Wheeler Compatibility Engine, Coupon Discount Calculator, Guest Checkout Handler, UPI QR Generator.
- **Template & API Pipeline:** Jinja2 server-side HTML rendering engine + JSON response serialisation for asynchronous requests.

Tier 3: Database & Persistence Layer (MySQL 8.0)

- **Relational Storage:** 14 normalized tables executing ACID-compliant transactional operations.
- **Query Interface (db_queries.py):** Modular database access layer using mysql.connector with connection recycling & parameterized queries.
- **Data Consistency:** Foreign key constraints cascading parent-child integrity across orders, order_items, and compatibility lists.

FUNCTIONAL DECOMPOSITION

Implemented System Modules

1. Product Catalog & Inventory

- **Scope:** 181 active automobile spare parts across 8 categories.
- **Features:** Search by part name, brand, part number; category filtering; price range slider; real-time stock levels.
- **Image Assets:** 159 verified real-world component photographs.

2. Two-Wheeler Fitment Engine

- **Compatibility Matrix:** 1,655 verified relations covering 11 Indian motorcycle and scooter platforms.
- **Live Client Search:** Instant search bar on product page filtering models in real time.
- **Badges:** Clear status indicators: Verified Fit, Universal Fit, Confirm Year/BS.

3. Customer Authentication & Portal

- **User Security:** Secure registration and login with encrypted password storage.
- **Account Hub:** Customer profile dashboard, order history, tracking status, and personal wishlist toggle.
- **Role Security:** Strict separation between regular customers and store admins.

4. Cart & Dynamic Checkout

- **Cart Management:** Real-time quantity modification, price computation, and empty cart handling.
- **Coupon Engine:** Discount coupon validation (e.g. FESTIVE10) with automatic order total deduction.
- **Guest Checkout:** Enables quick orders with recipient name, phone, and delivery address.

5. UPI QR Payment Workflow

- **Dynamic QR:** Generates order-specific UPI payment QR codes (Google Pay, PhonePe, Paytm).
- **Payment Verification:** Captures customer UTR transaction number for counter reconciliation.
- **Receipt Screen:** Instant order confirmation summary with itemized breakdown.

6. Administrative Control Hub

- **Inventory CRUD:** Add, edit, restock, or deactivate spare parts in the catalog.
- **Order Manager:** Update fulfillment status: Pending -> Confirmed -> Shipped -> Delivered.
- **Audit Logging:** Automated system_logs capturing critical administrative events.

DATA ARCHITECTURE

Database Design & Entity Relationship (ER) Schema

Catalog & Fitment Entities

• **categories:** PK: category_id
Fields: name, description
Stores 8 automotive categories.

• **products:** PK: product_id

FK: category_id -> categories

Fields: name, brand, part_number, price, stock_quantity, image_url, is_active.

• **product_compatibility:** PK: compatibility_id

FK: product_id -> products

Fields: vehicle_model, fitment_status, notes

Maintains 1,655 verified relations.

Orders & Commerce Entities

• **orders:** PK: order_id
FK: user_id (NULL for guest)
Fields: total_amount, payment_status, payment_method, order_status, shipping_address, created_at.

• **order_items:** PK: item_id

FK: order_id -> orders

FK: product_id -> products

Fields: quantity, unit_price, subtotal.

• **cart:** PK: cart_id

FK: user_id, product_id

Fields: quantity, added_at.

• **coupons:** PK: coupon_id

Fields: code, discount_percent, is_active.

User & Audit Entities

• **users:** PK: user_id
Fields: name, email, password_hash, role (admin/customer), phone, created_at.

• **wishlist:** PK: wishlist_id

FK: user_id, product_id

Personal saved items.

• **reviews & review_media:** Customer ratings & uploaded repair photos.

• **contact_messages:** Customer inquiries and fitment consultation queries.

• **system_logs:** PK: log_id

Fields: action, user, timestamp, details.

CORE OPERATIONS

CRUD Operations in Automobile Inventory Management

CREATE (Insert Record)

- **Admin Inventory:** Adding a new spare part into the catalog with part number, brand, price, and category.
- **Customer Order:** Inserting a new order header into 'orders' and line items into 'order_items'.
- **SQL Implementation:** INSERT INTO products (name, category_id, brand, price, stock_quantity) VALUES (%s, %s, %s, %s, %s);

READ (Query & Filter)

- **Vehicle Filter:** Fetching parts compatible with a user's bike (e.g. Hero Splendor Plus or Bajaj Pulsar).
- **Catalog Search:** Multi-parameter filtering by search keyword, category, price range, and in-stock status.
- **SQL Implementation:** SELECT p.* FROM products p JOIN product_compatibility pc ON p.product_id = pc.product_id WHERE pc.vehicle_model = %s;

UPDATE (Modify Record)

- **Stock Adjustment:** Decrementing available stock_quantity upon completed order placement.
- **Order Status:** Admin advancing fulfillment: Pending -> Confirmed -> Shipped -> Delivered.
- **SQL Implementation:** UPDATE products SET stock_quantity = stock_quantity - %s WHERE product_id = %s AND stock_quantity >= %s;

DELETE (Soft Deactivation)

- **Integrity Protection:** Physical DELETE cascades can break historical sales records and financial reports.
- **Soft Delete Approach:** We implement soft deletion by toggling 'is_active = 0', hiding it from catalog while preserving order history.
- **SQL Implementation:** UPDATE products SET is_active = 0 WHERE product_id = %s;

INTERFACE DESIGN

User Interface Design & User Experience (UX)

1. Home & Hero Banner

High-contrast automobile styling with immediate keyword search bar, popular brand showcases, and category shortcut tiles.

[Screenshot: Home & Hero Banner]

2. Catalog & Vehicle Filter Sidebar

Clean multi-faceted filter: category checklist, 11-model vehicle radio selector, and interactive price range slider.

[Screenshot: Catalog & Vehicle Filter Sidebar]

3. Product Detail & Fitment Guide

Displays high-res product photo, OEM specs, price, and our interactive live Two-Wheeler Compatibility Table with instant search.

[Screenshot: Product Detail & Fitment Guide]

4. Cart & Dynamic UPI QR Screen

Itemized cart breakdown, coupon input box, and dynamic UPI QR code generator enabling mobile scan-to-pay.

[Screenshot: Cart & Dynamic UPI QR Screen]

5. Guest Checkout Modal

Streamlined 1-step checkout capturing delivery address and phone number without requiring lengthy account creation.

[Screenshot: Guest Checkout Modal]

6. Admin Operations Dashboard

Tabular inventory overview, quick stock editor, order fulfillment dropdowns, and recent customer inquiry viewer.

[Screenshot: Admin Operations Dashboard]

DATA FLOW

System Workflow & Request-Response Lifecycle

Step 1: User Request	Client interacts with browser (e.g. filters by 'Hero Splendor Plus' or submits an order). An HTTP GET/POST request is dispatched to the Flask server.
Step 2: Routing & Middleware	Flask WSGI server inspects the URL route (@app.route('/products')), executes session security checks, and extracts query parameters.
Step 3: Controller & Business Logic	The controller calls db_queries methods. For fitment queries, it validates model parameters and prepares the SQL statement.
Step 4: MySQL Query Execution	The database connector opens a connection from the pool, executes parameterized SQL with bound parameters, and retrieves record tuples.
Step 5: Jinja2 Template Rendering	Flask merges the database recordsets with HTML templates (e.g. products.html), injecting stock counts, prices, and fitment badges.
Step 6: Client DOM Update	The browser receives semantic HTML/CSS and initializes JavaScript event listeners for live searching and quantity updates.

QUALITY ASSURANCE

Software Testing, Test Cases & Verification Results

TC ID	Module Tested	Test Description & Input	Expected & Actual Outcome	Result
TC-01	Fitment Engine	Filter catalog by vehicle model 'Hero Splendor Plus'.	Returns 164 compatible parts; completely excludes scooter CVT filters & clutch cables.	**PASS**
TC-02	Authentication	Attempt unauthorized access to /admin without admin session.	Flask session validator halts request and redirects to login with flash warning.	**PASS**
TC-03	Cart & Coupon	Apply coupon code 'FESTIVE10' on cart subtotal of ₹1,000.	Applies 10% discount (₹100); recalculates total to ₹900 accurately.	**PASS**
TC-04	Guest Checkout	Submit order with name, phone, and address without login.	Order created in database with user_id=NULL; unique order ID generated.	**PASS**
TC-05	Live Bike Filter	Type 'Pulsar' into live compatibility search bar on /product/24.	Instantly isolates Pulsar 150 & NS200 rows; hides non-matching models in DOM.	**PASS**
TC-06	Inventory Update	Complete checkout for 2 units of item with stock_quantity=10.	Atomic transaction completes; stock_quantity correctly decremented to 8 in MySQL.	**PASS**

SYSTEM EVALUATION

Practical Advantages & Current System Limitations

Practical Advantages

- **Zero Part-Mismatch Returns:** The verified two-wheeler compatibility engine eliminates customer confusion and dramatically reduces product return costs.
- **Frictionless Sales Conversion:** Guest checkout allows customers and local mechanics to complete purchases in seconds without password hurdles.
- **High-Performance Lightweight Stack:** Python Flask delivers lightning-fast page response times compared to heavy enterprise frameworks.
- **ACID-Compliant Transactions:** MySQL ensures consistent inventory numbers and prevents phantom reads or duplicate order anomalies.
- **Responsive Cross-Platform UI:** Optimized for mechanics ordering via Android smartphones or desktop counter staff.

Current Limitations

- **Manual Payment Reconciliation:** UPI payments rely on customer UTR transaction entry rather than an automated bank webhook settlement gateway.
- **Offline Barcode POS Integration:** The current web app does not yet interface with physical counter handheld laser barcode scanners.
- **Single-Currency Scope:** Transactions and pricing are restricted to Indian Rupees (INR) for domestic automobile retail.
- **Manual Stock Refill Trigger:** Store managers must periodically check stock reports rather than receiving automated supplier purchase orders.
- **Local Delivery Radius:** Shipping calculations currently assume local Hubballi and regional Karnataka fulfillment.

PROJECT ROADMAP

Proposed Future Enhancements

F1

1. Payment Gateway Webhook Integration

Integrate automated Razorpay or Cashfree webhooks to achieve instantaneous, zero-human-touch UPI payment confirmation and invoice clearing.

F2

2. Automated WhatsApp & SMS Notifications

Connect Twilio / Gupshup WhatsApp APIs to send instant order confirmations, dispatched courier tracking links, and pickup OTPs to customers.

F3

3. Automated PDF Tax Invoice Generation

Implement ReportLab / WeasyPrint to auto-generate downloadable, GST-compliant tax invoices with shop registration numbers upon order completion.

F4

4. Predictive Low-Stock Supplier Alerts

Establish automated background cron jobs that detect when fast-moving items drop below threshold and auto-draft supplier purchase orders.

F5

5. Machine Learning Part Recommendations

Deploy an association-rule recommendation engine suggesting complementary spares during checkout (e.g. suggesting brake fluid when purchasing disc pads).

PROJECT CONCLUSION

Conclusion & MCA Learning Outcomes

Project Conclusion & Impact

- **Successful Digitalization:** NK Automobiles successfully bridges traditional retail automobile shopkeeping with modern, scalable web engineering.
- **Solves Core Industry Pain:** The Two-Wheeler Compatibility Engine sets this system apart by eliminating part mismatch returns across 1,655 vehicle relations.
- **Commercial Readiness:** Features like Guest Checkout and dynamic UPI QR codes make the application instantly deployable for real-world business.
- **Clean Maintainable Codebase:** Built with clean MVC separation, parameterized SQL security, and modular Python Flask architecture.

MCA Technical Learning Outcomes

- **Full-Stack MVC Mastery:** Gained deep practical expertise connecting Python Flask controllers with dynamic Jinja2 templates and relational databases.
- **Relational Database Optimization:** Designed a 14-table 3NF schema executing ACID-safe transactions, indexing, and foreign key constraints.
- **Application Security:** Implemented password encryption (Werkzeug), role-based authorization, and SQL injection prevention.
- **Test-Driven Engineering:** Practiced automated test suite verification using Python unittest and live browser validation.

Thank you to our Project Guide, HOD, Faculty of Chetan Business School, and External Examiners. We welcome your questions!

VIVA PREPARATION

Appendix: Top 10 MCA Viva Voce Questions & Model Answers

Q1: Why did you choose Flask over Django?

Flask is a lightweight micro-framework giving full architectural control without the monolithic overhead of Django, making it ideal for custom automotive schemas and fast execution.

Q2: How do you prevent SQL Injection?

Every database interaction in db_queries.py uses parameterized queries (e.g. cursor.execute(query, tuple(params))) where user inputs are treated as literal values, never concatenated.

Q3: How is password security managed?

Passwords are never stored in plaintext. We utilize Werkzeug Security's generate_password_hash() and check_password_hash(), utilizing salted cryptographic hashing.

Q4: What is the normalization level of your schema?

The database is normalized to Third Normal Form (3NF). Repeating groups are eliminated, partial dependencies are removed, and transitive dependencies are decoupled into separate tables.

Q5: How does the Two-Wheeler Compatibility Engine work?

It uses a many-to-many relationship table (product_compatibility) linking products to specific bike models. A client-side JavaScript engine provides instant real-time filtering without page reloads.

Q6: How are database transactions handled during order checkout?

We wrap order creation and stock decrement in an atomic transaction: if any insertion fails, conn.rollback() triggers, preventing phantom orders or corrupted stock counts.

Q7: Why implement Guest Checkout?

Automobile customers often need quick emergency spares and abandon purchases if forced to register. Guest checkout maximizes conversion by requiring only recipient details.

Q8: How does the dynamic UPI QR code function?

It dynamically encodes standard NPCI UPI URI strings containing the payee VPA, payee name, amount, and order reference ID into a scannable QR code.

Q9: What happens if an admin tries to delete a product already present in previous orders?

We implemented Soft Deletion (setting is_active = 0). Physical deletion is avoided to protect referential integrity and historical financial reports.

Q10: How are user sessions handled in Flask?

Sessions are cryptographically signed using Flask's SECRET_KEY and stored in secure browser cookies, preventing client-side tampering of user IDs and admin roles.